IIT Madras

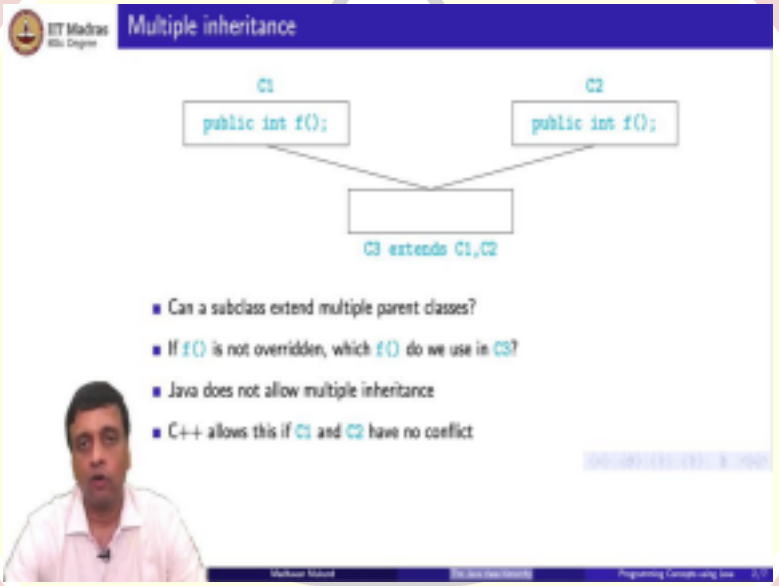
ONLINE DEGREE

Programming Concepts Using Java
Online Degree Programme
B. Sc in Programming and Data Science
Diploma Level
Prof. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute

The Java Class Hierarchy

So, we have seen that you can define subtypes that extend the parent type. So, how do these classes fit together because you have some classes which are above other classes? So, what is the structure of this java class hierarchy?

(Refer Slide Time: 00:28)



So, for example is it possible to extend 2 classes. So, can I define a class c3 which extends both c1 and c2 right. So, it inherits capabilities and instance variables and functions from both c1 and c2. So, the problem with having this kind of multiple inheritance is that there might be some contradictory definitions in the 2 parent classes 2 or more parent classes.

Suppose for instance there is a function f here with an identical signature there. So, c1 defines an f c2 defines an f and supposing c3 has no f. So, if c3 does not have an f then it has to inherit this f from the parent class but now there are 2 contradictory possibly definitions of f for it to you. So, which1 do we use? So, this is not easy to resolve of course. Now this is possibly something you might think is easy to check but what if I continue and I have you know a further hierarchy above this.

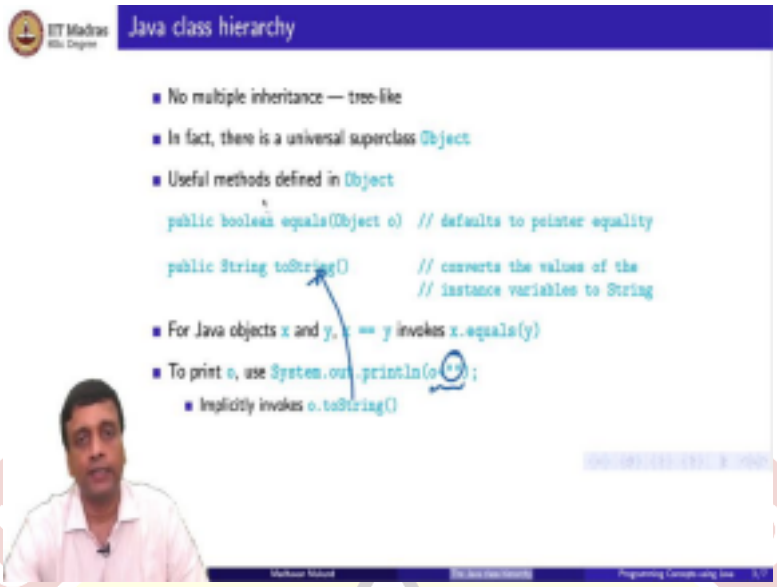
So, it is not in c1 or c2 but it is actually inherited by c1. So, this inheritance is actually transitive. So, if c3 something extends c3 some c4 it would also get a copy of everything that comes from c1 and c2. So, in java this is not allowed simply because of this possible contradiction right. Now it is eminently possible there is no contradiction that is everything in c1 is different from everything in c2 there is no function in c1 which has the same signature as c2 and vice versa.

That means that any function which I call implicitly through inheritance in c3 either uniquely come from c1 or from c2. But even if this possibility exists java does not allow it right. So, java does not allow multiple inheritances at all. On the other hand other languages like C++ do allow it if there is no conflict that is if c1 and c2 do not have any such clashing signature then c1 and c2 can be allowed.

So, this is a design choice we will see later that there are situations where you would like actually to inherit more than 1 because in a sense of class encapsulate sometimes a property. So, we discussed earlier about the property of being able to compare to all this. So, I would like to give some capability to a class saying that it inherits a comparison function from a class called comparable but then if it inherits that it cannot inherit anything else because of Java's lack of multiple inheritances.

So, we will see later how java allows us to get around this particular bottleneck but right now we cannot extend 2 classes.

(Refer Slide Time: 03:11)



So, that is a syntactic restriction of the java. So, since I cannot extend 2 classes. So, if I start from here then maybe I can extend it maybe I can extend it more than one way then these can again be extended and extended in more than one way. So, you can see that if I start from a class and look at all the subclasses they can form a tree. Now of course in principle I could have another class.

So, I could have here for example a class called employee which has a manager and maybe it has president and vice president and also to even below that and maybe separately I have a class which maybe stores dates right. And there is no connection obviously between dates and employees we also looked at you know an order processing example. So, supposing I have orders here and items and so on right.

So, there is no possible connection between them. So, they could be a lot of such things floating around but in general wherever I define subclasses I get tree. So, java goes a little further than this and it says actually this whole thing every class that you write in java sits inside a single tree and the way it does this is creates a root which is implicitly above every classic right.

So, you do not have to say when I say class employee it implicitly extends this built in class called object. Now this one might argue is not a great choice of name because object is a concept and now we are using object as a specific name of a class but we have to live with this or java has chosen this capital object as the name of this super class. So,

it is the universal super class it is a parent for the ancestor of every class that you ever use in java.

So, all your other classes are sitting below this. So, you might introduce employee here but it is connected to object and you do not have to say this right you do not have to say employee extends object it is implicit. So, object having an object that that route allows us to define some methods which are available implicitly to every class that we define. So, remember that inheritance works that way right if I have a class which extend something and I do not define a function in the in the child class the function that is there in the parent class will implicitly come down through inheritance.

So, there are some functions which are useful to have. So, if you remember when we wrote python classes we had to write these functions str in order to convert an object into a string. Now that we have this universal object capital O object class in java it has its own function called toString right which will convert the instance variables to a string. So, any class that you define implicitly has available to it this object copy of toString.

You do not have to write it yourself unless you want to change the way you want to do it of course which we will talk about override. Similarly there is an equals method defined an object. So, what is equals method does is it compares one object to another object. So, typically you will use it in this way O1 dot equals and what this equals method which is defined in this class object does is checks whether O1 and O2 actually point to the same object. So, this is what you might call pointer equality or reference equality.

That is O1 and O2 are actually so it was what python is called is right l1 is l2 are they both different names of the same memory location. So, it turns out that this is actually what is called implicitly when you use "= =" for objects. So, if x is an object and y is an object for example x may be an employee and y maybe a manager as defined but if I had written something like employees x is equal to new something.

Then I written manager y is equal to x and assuming that this is a legal assignment that is actually a manager type at this point okay. Then if I check if x is "= =" to y the answer will

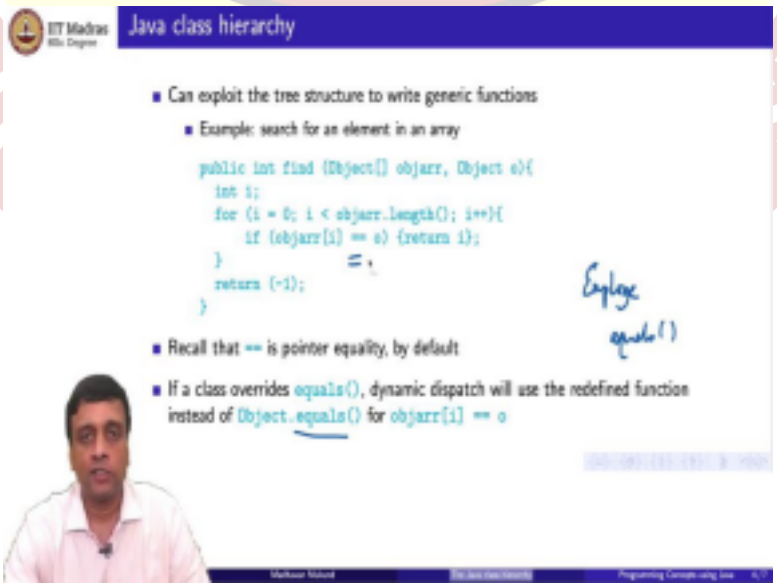
be true because it will call this and they will it will say that they both point to the same thing right. So, "= =" to when I compared to objects for equality in java the default is it goes and refers to the equals that is given in the object class.

So, that is the translation. Similarly when I try to print something if I want to convert it to a string now in the jar in the python context print automatically converts right. Here the way to convert is to put it an expression with an empty string. If I try to concatenate something to an empty string then java will try to type convert the previous part of the thing also to a string to make this plus work.

And when it does a type conversion to a string this implicitly invokes this toString(). So, I just have to say if I want to print an object O in some sensible way I just have to say O plus empty string. So, the O plus empty string will implicitly call O.toString() convert O also into a string and then concatenate it of course the empty string will contribute nothing to the output. So, I will get a representation of O as defined an object.

So, it is quite convenient this way that java has this one super parent class object okay. It is a kind of ancestor of every object in the hierarchy in which these kind of default functions are define so that they are automatically available to every class you define you do not have to keep defining these basic functions again.

(Refer Slide Time: 08:34)



So we talked earlier about this notion of generosity. So, everything can work. So, here is a

kind of generic find. Supposing I want to write a function which will take an array of any type of object and another object of that type and check whether the object I am looking for is there in the array or not right. So, fine takes an array and an object and checks whether O belongs to the object.

So, this is just a simple loop I just run from zero to the length of the array and I keep checking for each of them whether the array at position i in my array elementary position i in my array is O or not if it is I return that position if I run through the entire loop and I do not find it then for instance I could return -1 to indicate that there is no valid position. So, this is just a simple thing.

But the point is that this now will work for any type of object. So, if this happened to be an array of employees right and this is also an employee then comparing one employee with another employee will implicitly check whether. So, in this case equality means remember object equality they actually pointing to the same memory location. So, it will work whether these are employees or these are days. So, these are orders or these are items or anything. So, it will work.

So, long as this array is compatible with that array it will make sense and all of them are compatible with object. So, remember that when I have a something which expects an employee right and I feed it manager this is like an implicit assignment. So, just like we said that we can say Employee e = new Manager(). So, this is wherever a variable is of a higher type it can be instantiated to an object of a child type.

So, in the same way when I have a parameter of a higher type I can pass it a parameter which is compatible with it which is more specific. So, object is the least specific object is grandparent or an ancestor of everything any type I pass here will work and this is now actually separate. So, what I said earlier was that ideally you would like this object to be that; but this function cannot enforce that.

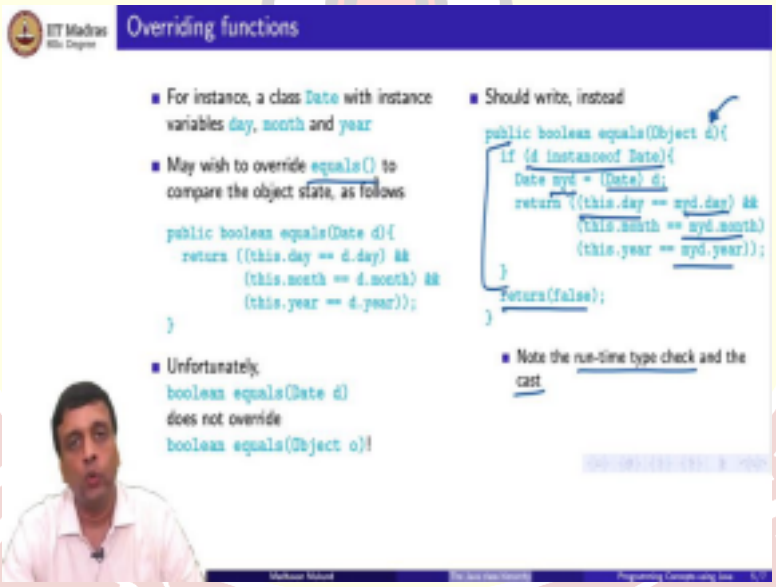
So, I could for instance pass a date here and have an employee array over there but that would not matter because it will just mean that this equality will not work. But there is no way that a

variable of type date can point to the same memory location as a variable of type employee. But this is a kind of example of the type of function that you can use which exploits this tree like hierarchy with object in this capital O object class sitting at the root.

So, now what if we actually had overridden this equals? So, supposing inside employee right I had written my own equals which overrides the object equals. Then what will happen is that at dynamically at runtime the dynamic dispatch will kick in. So, this double equality will call the dot equals but the dot equals will be that which is most specific to the object at the run time.

So, then I will actually get an equality comparison as defined saying employee rather than the one which is the pointed equality given an object. So, I could for instance have an equals function which compares the values of the instance fields and decides that 2 employees are equal if they have the same name and the same salary.

(Refer Slide Time: 12:18)



So, let us for instance consider a date. So, date as we normally have it has a day month and year. So, you might want to say that I want an equals function for date which will check whether this day is equal to the input. So, I remember that all these things are so yeah it is a symmetric thing right. So, d1 equal to d2 but when I say d1 equal to d2 it will be d1 dot equals d2. So, there will be an object within which equals is called and an object that is passed to it.

So, the object that is called is d is this and the object that is passed to it is d and I want to

check that the instance variables of the object that is called that this variables are equal to those who are coming in. So, this.day == d .day, this.day == d .month and this.day == d.year if all these things are true then the

answer is true. So, now I am not ready no longer demanding that this object and d are pointing to the same location in memory but rather that they have the same state remember the state of a variable is the current settings of all the instance fields in that object.

So, the question is if I want to do this then is this going to work or not. Now the unfortunate problem is it does not work because I have a signature here and which is not the same as a signature there. So, in order to override a function I must actually have the same signature. When we did the earlier example of overriding in the earlier case we had a signature in both cases for the manager and the employee which said double bonus float something.

So, this double and float with the same in both. So, the manager bonus was shadowing in some sense or making invisible the employee bonus. Now here this does not cancel out that right. And the problem the reason that this is a problem is because in the loop that we had before right I only have objects. So, when I use equal to equal to here I am actually asking for the equals where the parameter is an object type because it is going to be objarr[i].equals (O) but this is an objective right.

So, it will only work for the object. So, in this capital context it will work only for the object. So, by using that if I pass a date here it is not going to use the date function because the date function has a different signature. So, it will go back and unfortunately look at the old object type. So, how do we get around this. So, the way to get around this is to actually first we have to override this function.

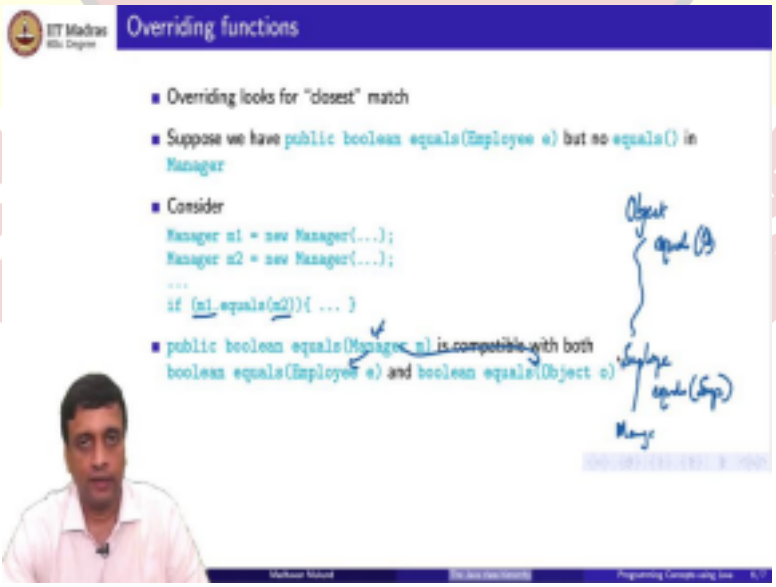
So, inside data we have to write a boolean equals (Object o) but then if it is an object I cannot be sure that is a date. So, I cannot start meaningfully talking about the instance variable. So, I have to do something like this I have to take a date in check that it is a date. If it is a date then I have to cast it. So, now I have a different variable with the type data of course I do not have create a new variable but it is needed to write it this way I can put a cast in each of these before the d and do it.

And then I can check whether this. So, this object is presumably a date otherwise I will not be working this. So, this.date is the same as the converted input d.date. So, I first verified that the input d can be converted to a date and then I have explicitly cast it to make sure that these things are now legal as far as the compiler goes because remember if I do not do that then the compiler would not allow me to access these instances where it is not.

We saw that earlier when an employee e was pointing to a manager I could not call the secretary just because I knew that it was a manager at the time I took cast it as a manager and then only call it. So, I have to cast it as a date and then we call it. So, you have to have a runtime check to make sure that the type is correct and if the type succeeds then you do a cast. And of course if this whole thing does not work that is this is a date and the input value is not a date then I just can assume that they are not equal I mean I do not have to worry because they cannot be the same type of object at all.

So, there is no question about the instance feels making or the state I think. So, I just throw it. So, this now becomes a valid way of overriding the equals and object in the context of that loop that we wrote before.

(Refer Slide Time: 16:53)



So, let us look at this overriding a little bit more carefully. So, overriding looks for the closest match in that given context. So, supposing we had defined this Employee e. So, this is like in date defining an equal it takes only a date as an argument. So, we have defined equals in

employee which takes employee as an argument but we have not defined equals and manager right. So, now I run this code.

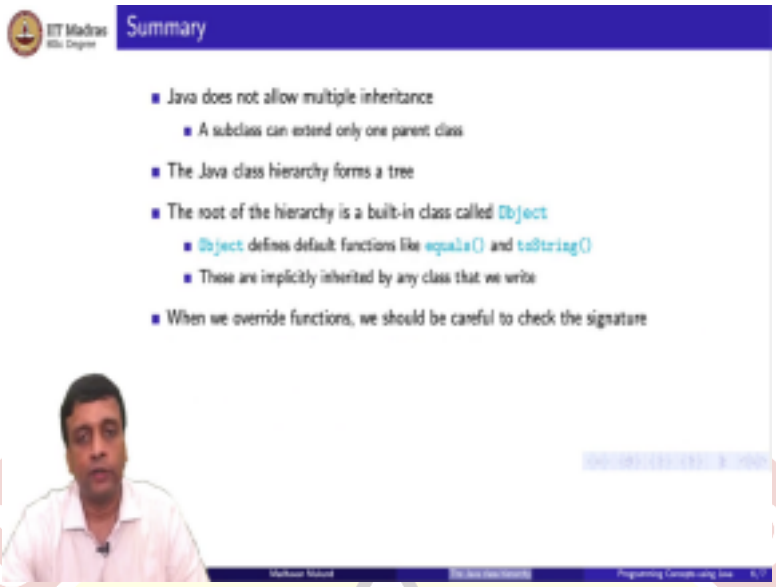
I create 2 manager objects m1 and m2 and then I ask whether m1 is equal to m2. So, now the question is what equals is going to be used here because I have at the top I have an object class which has this equals with object okay and somewhere down here I have employee which has an equals with the employee okay now below this I have manager okay. So, this manager equals remember that manager right it can go here and it can go here.

Because as far as the passing the argument it is like assigning a value right assigning available to a value. So, if the value is compatible that is if it is that type or lower in the hierarchy then I can assign. So, this call is compatible with both the question is what does overriding guarantee us in this case okay. So, it turns out that I actually we will get this one. So, it is like you look up the hierarchy from where you are to find out which are all the functions which are compatible with the call you have just made and you get the nearest one.

So, you should not confuse this example with the earlier example the earlier example said that the reason that that date thing did not work is because it was used in a loop right. When I used that date thing it did not work this way because I was using it in this loop right this loop here where both of them were statically of type object. So, in the context of type object it was not asking for a function whose argument is a date.

And that is why the date version did not work whereas here I am calling it in the context where both of them are managers. So, I know that I want to call manager with an input of type manager. So, then it will look for the nearest match going up the tree. So, this will actually use the boolean equals employee and not the bullion equals object which is there in the object class.

(Refer Slide Time: 19:31)



So, to summarize what we have seen is that Java does not allow multiple inheritance. So, subclass can extend only one parent class and that is because in general if you have multiple inheritance you could have contradictions you could have different functions with the same signature coming from 2 different sites and then the compiler does not know which one to take in case you are not overridden it in case if you are going to inherit which one do you inherit.

So, a consequence of this is that any class and subclasses will form a tree because you can always branch out but you cannot come back. And in particular what Java does it makes the entire thing into a single tree by creating this root class called object. So, right at the top of the tree there is a predefined class called object which is implicitly sitting above every class you ever define. So, you do not have to explicitly extend your class it implicitly extends object.

So, object has these clever built-in functions equals and toString and all that which allow you to do quality check and converting to string automatically but you are forced allowed to override them provided you overwrite them correctly if you do not override them these functions are implicitly inherited by every classes. But when we do overriding we have to be a bit careful to understand how and when this overriding works because it depends on the signature right.

It depends on the signature of the function that you are calling based on the current context of

the call. So, if you are calling it in one context it may call one thing you will be calling a different context. So, that is what we saw if the argument is an object it calls something else if the argument is a manager it called something else. So, you have to be a little careful to check that the overriding that you intend is actually the one that java sees.

Otherwise java might pick up the worst case it will always pick up the equals from object for example if you are using overriding for equals. But in general you should be a bit careful about checking the types and the signature when you are overriding.